

## Introduction

This first lab is simply to introduce you to the software that we will use for the remainder of the labs. You don't need to hand in a formal lab write-up for this lab. However, you should complete the exercises and make sure that you understand the material. If you have any questions, please ask.

We use the Matlab software package. Matlab is a really easy to use programming language with tons of built-in functions, including a bundle of plotting routines. If you already have some programming experience, this lab will be easy. If you have never programmed before, then this lab will provide a gentle introduction.

If you know Python, notice that Matlab syntax is very similar to Python's NumPy library. If you know C, notice that Matlab syntax is very similar to C syntax. If you know neither, then this lab will provide a gentle introduction.

The reason for using Matlab instead of numpy is that Matlab is clearer for signal processing. A large number of signal processing functions are built into Matlab, and the syntax is more clear than numpy. For example, the Fourier transform in Matlab is 'fft' while in numpy it is 'numpy.fft.fft'. The Matlab syntax is much easier to read.

Another reason for using Matlab, is that occasionally we will use the CREWES toolbox, which is a collection of Matlab functions for seismic processing. Many of the functions we will develop in this course have some similar version in the CREWES toolbox. You can find the CREWES toolbox at: '<http://www.crewes.org/ResearchLinks/Software/MatlabToolbox/>'.

Similarly, Matlab has a large number of built-in functions for signal processing, including the Fourier transform, filtering, and plotting. You can find the Matlab documentation at: '<http://www.mathworks.com/help/matlab/>'.

NOTE: you can use the crewes toolbox to check your results or get ideas for your own code, but you should write your own code for the lab exercises.

On the other hand, if you prefer to use Python, please feel free to do so. The lab exercises can be completed in either language. If you choose to use Python, please make sure that you have the numpy and matplotlib libraries installed.

## Manipulating matrices, vectors and scalars

Matlab stores information in *variables*. Almost every variable in Matlab is stored as a *matrix*. Try entering the following commands (without the bracketed and italicized comments, or the command prompt, `>>`)...

1. `>> a = 2` (*assigns 2 to a*)
2. `>> x = [1;2;3]` (*assigns a column vector to x*)
3. `>> y = [4,5,6]` (*assigns a row vector to y*)
4. `>> z = [0:2:20]` (*assigns a row vector  $\begin{bmatrix} 0 & 2 & 4 & \dots & 20 \end{bmatrix}$  to z*)
5. `>> A = [1,2,3;4,5,6;7,8,0]` (*assigns a matrix to A*)
6. `>> x` (*prints x*)
7. `>> whos` (*prints a table of stored variables*)
8. `>> size(A)` (*prints the size of A*)
9. `>> size(z)` (*prints the size of z*)
10. `>> length(z)` (*prints the length of z*)

'a' is a scalar ( $1 \times 1$  matrix); 'x' is a vector ( $3 \times 1$  matrix); 'y' is a vector ( $1 \times 3$  matrix); 'A' is a matrix ( $3 \times 3$  matrix). `>>` is called the command prompt.

You can access subsets of the matrices and vectors. Try the following...

1. `>> x(2)` (*access the 2nd element of the vector*)
2. `>> x(2:3)` (*access the 2nd through 3rd elements of the vector*)
3. `>> A(2,3)` (*access the element,  $A_{23}$  of the matrix*)
4. `>> A(1:2,2:3)` (*access the elements  $A_{ij}$  for  $i = 1 \dots 2, j = 2 \dots 3$* )

You can stitch vectors and matrices together in any logical order. For example, try the following...

1. `>> [x',y]` (*attach y to the tail of  $x^T$* )
2. `>> [A,x]` (*attach a column to the right side of A*)

## Some Linear Algebra

Matlab supports pretty much every operation that you can find in your linear algebra textbook (Here we look at just a few simple examples). Try the following...

1. `>> A'` (*returns the transpose of  $A$* )
2. `>> A*x` (*matrix-vector multiplication of  $A$  and  $x$* )
3. `>> x+y'` (*vector addition of  $x$  and  $y^T$* )
4. `>> x.*y'` (*element-wise multiplication of  $x$  and  $y^T$* )
5. `>> x./y'` (*element-wise division of  $x$  and  $y^T$* )
6. `>> a*x` (*multiplication of a vector by a scalar*)
7. `>> x+a` (*vector addition of  $x$  and a vector of the same size with elements  $a$* )

Of course this is just a short list. If you have time, you can look at the on-line documentation for more features.

## Plotting

Matlab has some easy-to-use and useful plotting routines. In this lab we will use the 'plot' routine. To learn about other plotting functions in Matlab you can use the 'help' command. Try the following...

1. `>> help plot`
2. `>> help stem`
3. `>> help subplot`

You don't have to read the instructions... Just keep in mind that you will have to use these plotting routines for every lab exercise, including this one.

## Some Computer Science

Matlab is a fully featured *scripting* language supporting functions, loops, and conditional statements. This means that commands can be typed into a text file which can be interpreted by Matlab.

- open a new text file in a text editor called 'my\_first\_script.m'

- In Matlab change directories to the one that contains your file by typing, at the Matlab prompt, ‘cd path/to/file’. This will ensure that Matlab can see your text file. If you are not familiar with the ‘cd’ command, please ask.

We will enter commands into this text file. For this first example, our objective is very simple:

- Write some computer code that will produce two vectors,  $\mathbf{x}$  and  $\mathbf{y}$  such that  $\mathbf{y} = \sin(\mathbf{x})$ . Instructions follow...

## looping

enter the following text (code) into ‘my\_first\_script.m’

---

```
% Example of a Matlab script
% A MATLAB script is a plain .m file containing a sequence of commands
% that run in the current workspace (no inputs or outputs). Use scripts
% to automate tasks, run procedures, and create simple workflows.
% everything following the percent sign is a comment; it does nothing.

n=101; % the semi-colon forces Matlab to NOT print the value of n.
x=zeros(n,1); % create a vector of zeros with n rows and 1 column
y=zeros(n,1); % create a vector of zeros with n rows and 1 column
for ii=1:1:n % equivalent to: for ii=1:101
    x(ii) = -2.0*pi + 4.0*pi*(ii-1)/(n-1);
    y(ii) = sin(x(ii));
end

plot(x,y);
```

---

‘ii’ is the loop index; it is set from the elements of the row vector:  $\begin{bmatrix} 1 & 2 & \dots & 100 \end{bmatrix}$ . Matlab *loops* through the elements of this vector, executing the contents of the loop (the lines between ‘for ii=1:1:100’ and ‘end’) for each value of ‘ii’. You can run this code by entering the file name (without the suffix) at the command prompt. Make sure you try this to see if your code works. You should see a window pop up with a plot of your curve (exciting, eh?).

Alternatively, you can use the ‘while’ looping command. To learn more about the ‘while’ looping command, use the already mentioned Matlab help feature.

- Try editing your ‘my\_first\_script.m’ file, replacing the ‘for’ loop with a ‘while’ loop.

Finally, it should be noted that the above code can be written without using a loop. An idea which you will need shortly.

## conditionals

Create a second file called ‘my\_second\_script.m’ and add the following code. The ‘if’ statement is the conditional. If the condition ‘ $x(ii) < 0$ ’ is true, the statement, ‘ $y(ii) = -x(ii)$ ’ is executed. Otherwise, ‘ $y(ii) = x(ii)$ ’ is executed.

---

```
% everything following the percent sign is a comment; it does nothing.

clear; % clears all variables from Matlab's memory.

n=101; % the semi-colon forces Matlab to NOT print the value of n.
x0=-2.0;
T=abs(2*x0); % abs(c) returns the absolute value of c.
x=zeros(n,1); % create a vector of zeros with n rows and 1 column
y=zeros(n,1); % create a vector of zeros with n rows and 1 column
for ii=1:1:n % equivalent to: for ii=1:101
    x(ii) = x0 + T*(ii-1)/(n-1);
    % Conditional: the if/else checks whether x(ii) is negative.
    % If true, it assigns y(ii) the negated value (-x) to compute |x|.
    % Otherwise, it assigns y(ii) the original value to compute |x|.
    if x(ii) < 0
        y(ii) = -x(ii);
    else
        y(ii) = x(ii);
    end
end

plot(x,y);
```

---

Try running this new script (wow, even more exiting!).

## writing functions

So far we have written two Matlab scripts. But, what if we want to use the script over and over again with different values for  $n$ ,  $T$  and so on. This is what functions are for. Each function must be in its own file. The first line of code in the function file will always look like,

$$\text{function}[\text{return\_val1}, \text{return\_val2}, \dots] = \text{my\_function}(\text{arg1}, \text{arg2}, \dots)$$

This line is called the function header. The function’s name is ‘my\_function’, and the function must be written in a text file called ‘my\_function.m’. The remainder of the code in the function file will use the variables: ‘arg1’, ‘arg2’, ...; will assign values to variables: ‘return\_val1’, ‘return\_val2’, ...; and will look similar to the code in the scripts that you have already written.

## Exercise

In this exercise you will write a function for the code in ‘my\_second\_script.m’. Once again, your goal is to write code that computes the vectors  $\mathbf{x}$  and  $\mathbf{y}$  such that,

$$y = f(x) = \begin{cases} -x & , \quad x < 0 \\ x & , \quad x \geq 0 \end{cases} \quad (1)$$

- Open a new file called ‘saw\_tooth.m’
- Use the following function header:

function[x,y] = saw\_tooth(n,x0)

- The function should NOT plot  $y(x)$ . It should only compute  $y$  and  $x$ .

The body of the function is for you to write. Of course it will look very similar to ‘my\_second\_script.m’; however, I am placing the following restriction on your code:

YOU MAY NOT USE LOOPS OR AN INDEX VARIABLE SUCH AS ‘ii’!

Once you’ve written your function, make sure it works! Try using it. For example, try:

1. `>> [x,y] = saw_tooth(101,-2);`
2. `>> plot(x,y)`

Finally, let’s put your function to use inside a Matlab script. In this script you will plot your function and an approximation to that function using sine and cosine functions (ie. a Fourier series) such that  $f(x)$  in equation (1) is approximated as,

$$f(x) \approx \frac{a_o}{2} + \sum_{m=1}^M \left[ a_m \sin\left(\frac{m\pi x}{2}\right) + b_m \cos\left(\frac{m\pi x}{2}\right) \right]$$

where

$$\begin{aligned} a_o &= 2 \\ a_m &= 8/(m\pi)^2 \quad \text{for odd } m \\ a_m &= 0 \quad \text{for even } m \\ b_m &= 0 \quad \text{for } m = 1, 2, \dots \end{aligned}$$

and  $f(x)$  is approximated and plotted for  $M = 1$  through  $M = 9$ . Detailed instructions follow...

- open a new file called 'fourier.m'.

Input the following code into the text file.

---

```
% fourier Demonstrate Fourier series approximation of a sawtooth/absolute-value waveform
% This script generates a sawtooth waveform using saw_tooth, then plots the
% original function and successive partial Fourier cosine-series
% approximations (odd harmonics) to illustrate convergence.
% Example: run the script to see plots for M = 1,3,5,7,9.
[x,f] = saw_tooth(101,-2);

subplot(3,2,1),plot(x,f),xlabel('x'),title('f(x)');

a0=2;

x_approx = [-2:.1:2];
f_approx = .5*a0*ones(size(x_approx));
for ii=1:1:5
    M = 2*ii-1;
    a = -8/(M*pi)^2;
    f_approx = f_approx + a*cos(M*pi*x_approx/2);
    t = sprintf('M=%i',M);
    subplot(3,2,ii+1),plot(x_approx,f_approx),xlabel('x'),title(t);
end
```

---

Run the script. If your typing skills are sufficient, you should see something interesting. Namely that the Fourier series works!

You can find some useful Matlab tutorials at: '<http://www.math.siu.edu/matlab/tutorials.html>'